

evopi Architecture

자기진화 하네스 구조 — Claude Code · oh-my-pi · prime-agent 와 비교

레포 — <https://github.com/sunwoo95/oh-my-evopi>

설치 — `curl -fsSL https://sunwoo95.github.io/oh-my-evopi/install.sh | sh`

oh-my-evopi 팀 세미나 · 2026-09

근거 : docs/analysis/*.md, docs/design/DECISIONS.md (파일 : 라인 인용)

참조 시나리오 : Claude Code Architecture (AWS Korea 최우형 , 2026)

Agenda

11 개 섹션으로 분해한 evopi 아키텍처 — Claude Code 8 레이어 맵과 같은 순서로

01	evopi 개요	prime 골격 + omp 자산 + Evo-Harness 델타 — 무엇을 왜 만들었나
02	전체 아키텍처 맵	Master Agent Loop 중심 8 레이어 + evo 레이어, Claude Code 맵과 대응
03	The Agentic Loop	Think → ipython 실행 → Verify, agentLoop + AgentSession
04	Models & Tools	9 API 종 · 단일 툴 ipython · IPython 커널 (uv + dill)
05	에이전트 구동 + Skills	5 가지 입력 파일, Python 스킬 계약, Harness 원장
06	Input Layer	인터페이스 6 종, curl 설치, 세션 (jsonl+dill), 권한 게이트 + 안전 기본값 (v0.10 · v0.11)
07	Knowledge Layer	시스템 프롬프트 조립, 하네스 주입 (MMR), 요약 컴팩션
08	Execution & Multi-Agent	sdk streamFn 파이프라인 (auth-pool·dialect), rlm 서브에이전트
09	Observability & Integration	확장 훅 31 종, 빌트인 3 종, MCP(커널 내), 프로바이더, natives
10	Evo Layer	논문 ↔ prime 델타 판정, grounded-refine(D1+D4), 4-arm 평가
11	비교와 점검	4 자 비교 매트릭스, 하네스 점검 (강점 · 리스크), End-to-End, 요약

01

evopi 개요

prime-agent 골격 · oh-my-pi 자산 · Evo-Harness 델타

evopi 개요

스스로 하네스를 다듬는 코딩 에이전트 CLI

SELF-EVOLVING HARNESS

evopi 는 prime-agent 의 골격 (RLM 하네스 + IPython 커널) 을 그대로 쓰고 , oh-my-pi 의 TypeScript 자산 (dialect · auth-pool · hashline · mnemopi) 을 선별 이식한 뒤 , Evo-Harness 논문 (arXiv 2608.15071) 의 접지 피드백 델타를 optional 레이어로 얹은 코딩 에이전트다 . 설정 경로는 ~/.evopi 하나 , 실행 커맨드는 evopi.

RLM-NATIVE

파이썬 커널이 곧 도구

내장 LLM 툴은 ipython 하나 . 셀 · 편집 · 서브에이전트 · MCP 를 커널 안 Python 호출 (rlm.bash, edit 스킴 , rlm()) 로 합성한다 .

SELF-EVOLVING

하네스 원장을 스스로 편집

prompt / memory / skill / subagent 4 종 엔트리를 refine 라운드가 create · update · delete. 롤백 스냅샷과 refinements.jsonl 이력 .

PROVIDER-AGNOSTIC

어느 모델이든 같은 루프

9 API 종 · OAuth 3 종 · Bedrock · Databricks · 오픈모델 in-band 툴콜 (dialect 11 종) · 다중 키 폴 로테이션 .

EVO OPTIONAL

evo off = prime 원본

EVOPI_EVO=off 면 grounded-refine · MMR 선택기가 로드되지 않아 prime 동작과 바이트 동일 — A/B 대조군이 공짜로 생긴다 .

1 tool

내장 LLM 툴 = ipython

4 kinds

하네스 엔트리 종류

31 hooks

확장 훅 이벤트

4 arms

A/B 평가 조건

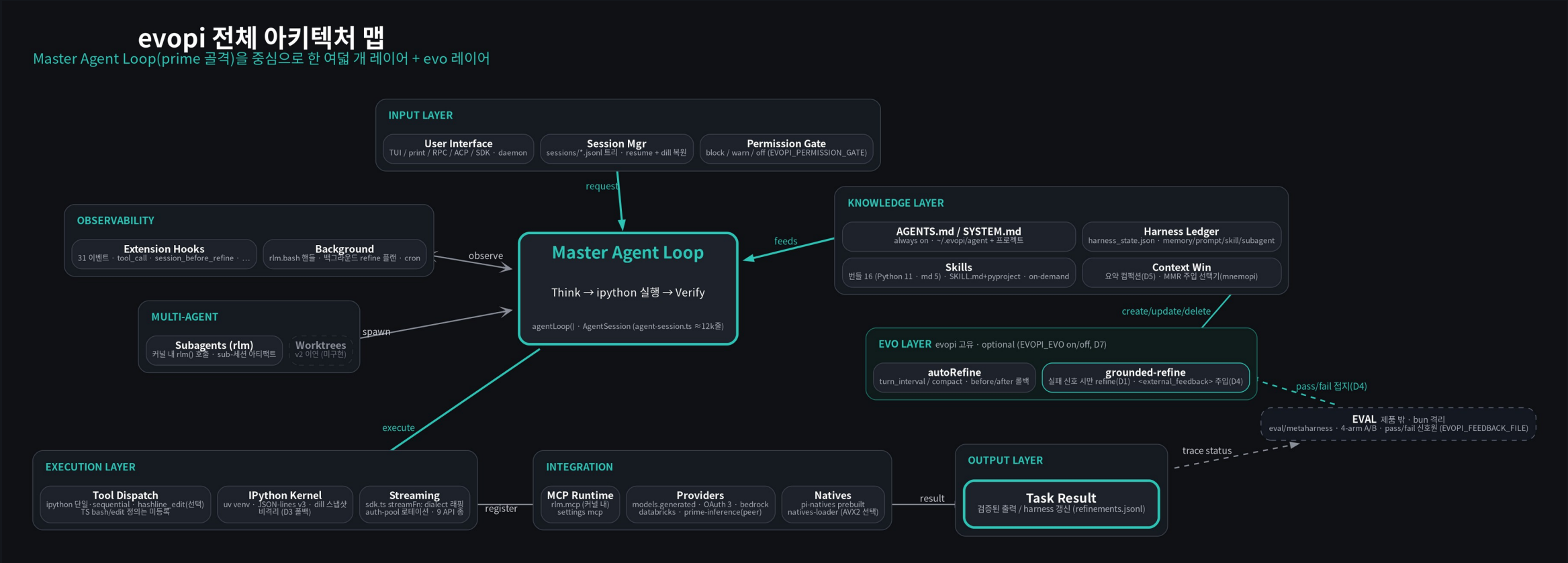
02

전체 아키텍처 맵

Master Agent Loop 중심 8 레이어 + evo 레이어

전체 아키텍처 맵

Master Agent Loop(prime 골격) 을 중심으로 한 여덟 개 레이어 + evo 레이어



docs/diagrams/evopi-master-arch.dot (graphviz) — 각 박스 부제는 코드 실측 (docs/analysis/evopi-harness-inventory.md)

참조 : Claude Code 전체 아키텍처 맵

같은 8 레이어 격자 — evopi 맵과 박스 단위로 대응된다



참조 슬라이드 (AWS Korea, Claude Code Architecture p.3) 전사본을 동일 스타일로 재현 — refs/claude-code-8layer-map.md

맵 대응표 — Claude Code ↔ evopi

박스 하나하나가 어디로 대응되는가, 그리고 evopi 만의 9 번째 레이어

	Claude Code 참조 맵	evopi 대응 구성요소	차이의 본질
INPUT	CLI/IDE/CI-CD · Resume/Fork · Ask/Allow/Deny	TUI/print/RPC/ACP/SDK/daemon · jsonl 트리 +dill 복원 · permission-gate block/warn/off	세션 복원이 대화 + 커널 변수 (dill) 까지 포함
KNOWLEDGE	CLAUDE.md · Auto Memory · Skills · Compaction	AGENTS.md/SYSTEM.md · Harness Ledger · SKILL.md+pyproject · 요약 컴팩션 +MMR	메모리가 파일이 아니라 편집 가능한 구조화 원장
EXECUTION	Tool Dispatch(typed) · Prompt Cache · Streaming	ipython 단일 톨 (sequential) · IPython 커널 · streamFn(auth-pool·dialect)	다수 도구 → 단일 REPL 셀로 합성
MULTI-AGENT	Subagents(isolated) · Worktrees	rlm() 커널 내 호출 → in-process 자식 Agent · Worktrees 는 v2	서브에이전트가 Python 함수 호출
OBSERVABILITY	Hooks(lifecycle) · Background	확장 훅 31 이벤트 · rlm.bash 핸들 /cron/ 백그라운드 refine	훅이 refine 플래너까지 교체 가능 (session_before_refine)
INTEGRATION	MCP Runtime · Ext Servers	rlm.mcp(커널 내) · 9 API 프로바이더 · pi-natives	MCP 도구도 Python 네임스페이스로 노출
OUTPUT	Task Result / Memory updated	Task Result / harness 갱신 (refinements.jsonl)	출력이 곧 다음 하네스 편집의 증거
(+) EVO	— (없음)	autoRefine + grounded-refine(D1·D4) · EVOPI_EVO 게이트 · metaharness 접지	Claude Code 에 없는 9 번째 레이어

03

The Agentic Loop

Think → ipython 실행 → Verify

The Agentic Loop

모든 작업을 관통하는 루프 — 도구가 하나라서 루프가 단순하다

CORE LOOP

agentLoop()(packages/agent/src/agent-loop.ts, 963 줄) 이 turn_start → 모델 스트리밍 → 툴 실행 → turn_end 를 반복한다. AgentSession(agent-session.ts ≈12k 줄) 이 그 바깥에서 시스템 프롬프트 재조립, 자동 컴팩션, refine 체크포인트를 감싼다.



steering

턴 중 지시 주입

follow-up

턴 종료 후 후속

compact

자동 압축 → refine 트리거

sequential

커널 단일 스레드 — 툴 병렬 없음

04

Models & Tools

추론하는 모델 · 행동하는 단일 툴 · 그 뒤의 커널

Models & Tools

루프를 움직이는 두 축 — 9 API 종의 모델과 ipython 하나의 도구

MODELS — 추론 엔진

pi-ai stream(): anthropic-messages · openai-completions/responses · azure · codex · google · vertex · mistral · bedrock. models.generated 카탈로그 + models.json 사용자 등록 . OAuth 3 종 , Databricks, prime-inference 는 peer.

TOOLS — 행동 수단

레지스트리 등록 툴은 ipython(기본) 과 hashline_edit(선택)
둘뿐 (tools/index.ts:59). TS bash/edit 정의는 있지만 미등록 — 셸과 편집은 커널 안
Python(rlm.bash, edit 스킬) 이 담당한다 .

IPYTHON

영속 REPL

단일 문자열 code 파라미터 .
sequential 실행 . 65,536 자
출력 캡 .

BASH

rlm.bash()

프로세스 그룹 저널링 , 핸들
(tail/poll/kill), 커널 종료 시
회수 .

EDIT

edit 스킬 · hashline_edit

Python edit 스킬이 기본 .
hashline 앵커 편집은 --tools
게이트 .

SUBAGENT

rlm()

커널에서 함수 호출 →
host_request → in-process
자식 Agent.

MCP

rlm.mcp

MCP 서버 도구를 Python
네임스페이스로 노출 . settings
로 등록 .

9

API 종 (register-builtins)

1 + 1

내장 툴 ipython + 선택 hashline_edit

11

in-band 툴콜 dialect

N keys

EVOPI_API_KEY_POOL_<PROVIDER>

IPython Kernel

prime 에서 무변경 이식한 실행 엔진 — 세션을 넘어 살아있는 네임스페이스

KERNEL CONTRACT

python -m rlm.repl 서버프로세스 . stdin 으로 JSON-lines 요청 , stdout 으로 이벤트 (ready/stdout/stderr/result/display/host_request/error/done),
프로토콜 v3 정확 일치 . 모든 요청은 단일 큐로 직렬화된다 (repl-manager.ts:769). 커널 = 단일 스레드 .

BOOTSTRAP

uv 부트스트랩

uv python install 3.11 → venv →
runtime + dill + 12 패키지 (전부 wheel,
R5) → Python 스킵 editable 설치 .
런타임 소스 해시가 바뀌면 venv 자동
재빌드 (bootstrap.ts).

REPL MANAGER

수명주기 1,502 줄

spawn → ready 30s → 셀 실행 →
interrupt(1s 후 호스트 측 강제 abort) →
shutdown(5s). 손상 프레임은 SIGKILL
후 스냅샷 복원으로 복구 .

SNAPSHOT

dill per-variable

성공 실행 1.5s 후 kernel-state.dill.json
저장 (256MB/16MB 상한 , 언피클 불가
이름은 skip 보고). resume 시 복원 →
모델은 변수를 계속 본다 .

BOOT GATE

부팅 세마포어

동시 커널 부팅 min(16, max(4, cpu×2))
로 제한
(EVOPI_MAX_CONCURRENT_KERNEL_
BOOTS). 실행 리소스 제한은 아니다 .

30s

ready 타임아웃

65,536

셀 출력 문자 캡

256MB

스냅샷 총량 상한

비격리

uid/rlimit/ns 없음 — D3 폴백

05

에이전트 구동 및 Skills

5 가지 입력 · Python 스킬 계약 · Harness 원장

에이전트를 움직이는 5 가지 입력

AGENTS.md / SKILL.md / Prompt / Hook — 그리고 Claude Code 에 없는 다섯 번째 , Harness 원장

FIVE INPUTS

Claude Code 의 4 파일 (CLAUDE.md · SKILL.md · Prompt · Hook) 이 그대로 대응되고 , 여기에 에이전트 스스로 편집하는 harness_state.json 이 추가된다 . 사람이 쓰는 지식 (AGENTS.md) 과 에이전트가 쓰는 지식 (Harness) 이 분리된다 .

MEMORY

AGENTS.md /
SYSTEM.md

~/.evopi/agent 와 프로젝트
.evopi/agent. 매 턴 시스템
프롬프트에 포함 .
APPEND_SYSTEM.md 로 덧붙임 .

SKILL

SKILL.md
(+pyproject)

md 스킬은 설명 매칭 시 본문 로드 .
Python 스킬은 커널에 import
되어 함수로 호출된다 .

PROMPT

Prompt

지금의 일회성 지시 . steering
으로 턴 중간에도 주입 가능 .

HOOK

Extension

jiti 로 무컴파일 TS 로드 . 31
이벤트에 핸들러 . tool_call 차단 ,
플래너 교체까지 .

HARNESS

harness_state.js
on

prompt/memory/skill/
subagent 엔트리 . refine
라운드가 편집 , 시스템 프롬프트에
kind 별 요약 주입 .

항상

AGENTS.md — 매 턴

매칭 · import

SKILL.md — 설명 / 커널

지금

Prompt

이벤트

Hook — 31 종

자동 진화

Harness — refine 라운드

무엇이 언제 컨텍스트에 들어오나

로딩 시점 · 트리거 · 갱신 주체 비교

	AGENTS.md 항상	SKILL.md 필요할 때	Prompt 지금	Hook 자동	Harness 자동 진화
로딩 시점	세션 시작 + 매 턴 재조립	md: 설명 매칭 시 본문 py: 커널 부팅 시 import	사용자 입력 시점	라이프사이클 이벤트	매 턴 요약 (kind 별 6 개) 주입
트리거	자동 (조건 없음)	description 매칭 / 코드 호출	수동 (사람)	tool_call, turn_end 등	turn_interval 25 · compact · 외부 실패 신호
갱신 주체	사람	사람 (+skill-creator 스킬)	사람	사람 (개발자)	에이전트 (refine) + 롤백
대표 예시	코딩 규칙 , 빌드 명령	edit · websearch · refine	" 이 버그 고쳐줘 "	permission-gate 차단	" 이 레포는 pnpm 사용 " 메모리

~/.evopi/agent	.evopi/agent	~/.agents/skills	harness/
전역 설정 루트	프로젝트 오버레이	벤더중립 스킬 경로 유지	전역 + 세션 로컬 원장

Skills — Python 스킬 계약

SKILL.md 오픈 표준 위에 pyproject 를 엮어 커널이 import 하는 스킬

SKILL CONTRACT

SKILL.md(프론트매터 name/description) + pyproject.toml + src/<import_name>/__init__.py. 스킬 로더가 SKILL.md 를 찾고 pyproject 가 있으면 kind:"python" 으로 분류 → uv 가 editable 설치 → 부트스트랩 코드가 import. 설치 실패는 경고만 (비치명).

```
packages/coding-agent/skills/edit/

SKILL.md           ( 필수 : 설명 · 사용법 )
pyproject.toml     (name, dependencies)
src/edit/__init__.py ( 커널이 import )

# 커널 안에서
from edit import apply_patch
apply_patch(path, diff)

# refine 스킬
await refine.run() # host_request → TS
```

name	스킬 식별자 — 프롬프트 스킬 목록에 노출
description	md 스킬 트리거 · 유효 YAML 필수 (따옴표)
pyproject	의존성 + 형제 스킬 로컬 의존 (위상 정렬 설치)
hash	pyproject 해시 일치 시 재설치 스킵
MIME	display: application/vnd.evopi.{diff,attachment,agent-message}

16	11 / 5	editable	3 경로
번들 스킬	Python / Markdown	uv pip install -e	번들 · ~/.evopi · 프로젝트 (+~/.agents)

Harness 원장 — 에이전트가 편집하는 지식

rlm.harness: 프롬프트 노트 · 메모리 · 스킬 서술 · 서브에이전트 명세의 영속 원장

CONTINUAL HARNESS

" 두 번째 실행 엔진이 아니다 " — 하네스는 실행 코드가 아니라 다음 턴을 위한 라우팅 · 컨텍스트 힌트다 . 세션 로컬 session-artifacts/<id>/harness/harness_state.json 과 전역 ~/.evopi/agent/harness/ 를 병합해 주입한다 .

PROMPT

prompt note

시스템 프롬프트에 덧붙일 짧은 규칙 . 예 :
" 테스트는 npnm test 로 "

MEMORY

memory

재사용 사실 . scope local(세션) /
global(전역) . mnemopi MMR 이 다양
성 · 최근성으로 선택 .

SKILL

skill 서술

Python 참조 (reference) 를 강제 —
존재하는 스킬만 등록 가능 (validateEdit).

SUBAGENT

subagent 명세

rlm() 으로 띄울 특화 에이전트 정의 .
list/delete 는 host_request.

create/update/delete

3 편집 연산 (MERGE/SKIP 없음)

25 턴 · 20 분

autoRefine turnInterval · cooldown

before/after

롤백 스냅샷 · refinements.jsonl

루트 세션만

_rlmDepth==0 + 영속 artifact 필수

06

Input Layer

인터페이스 · 설치 · 세션 · 권한

Input Layer — 인터페이스와 설치

어디서 실행하고 어떻게 들어오는가

SAME LOOP, EVERYWHERE

cli.ts → cli-main.ts → main.ts 하나의 진입에서 모드만 갈린다. 설치의 curl 원라이너가 GitHub Pages 의 릴리스 tarball(SHA256 검증) 을 받아 npm -g 로 설치하고, 필요 시 uv 부트스트랩까지 수행한다. 상태는 ~/.evopi/agent 한 곳.

TUI

대화형 터미널

pi-tui 기반. 스플래시 (EVO 워드마크),
/login · /compact · /refine 슬래시 명령.

HEADLESS

print · RPC · ACP · SDK

파이프 / 에디터 통합 / 프로그램 임베딩.
createAgentSession() 으로 동일 세션
생성.

DAEMON

goal · cron · heartbeat

슈퍼바이저 + 세션 워커
(AgentConnection 경계). 장기 무인 실행
— v1 동결.

INSTALL

curl | sh

install.sh 1,621 줄 : preflight →
tarball+SHA256SUMS → npm -g → (
옵션) uv/ 커널 부트스트랩. 실검증
0.11.0.

1 path

~/.evopi (.omp/.prime 잔존 0)

0.11.0

게시 버전 (GitHub Pages)

node ≥22.8

런타임 (Bun 실패 0 건)

evopi

bin 이름 · APP_NAME

Sessions & Permissions

세션 연속성과 2 계층 권한 모델 (D4)

INTENT vs ENFORCEMENT

의도 계층 (permission-gate 확장) 은 항상 로드되어 위험 명령을 차단한다. 집행 계층 (OS 샌드박스) 은 능력 프로브가 통과할 때만 활성화 — 현 환경은 unprivileged usersns 불가로 bwrap 실패 → 컨테이너 경계가 집행 계층을 대신한다 (D3 폴백).

SESSIONS

jsonl 트리 + dill

sessions/<id>.jsonl 대화 트리, session-artifacts/<id>/ 에 kernel-state.dill · scheduled-jobs · harness.resume 시 변수까지 복원.

PERMISSION GATE

block / warn / off

19 위험 패턴 + rm 대상별 판정, bash()/subprocess 셸 마커 5 종, 보호 경로 8 종 쓰기 확인. UI 있으면 Yes/No, 없으면 즉시 block. EVOPI_PERMISSION_GATE → 다음 슬라이드.

SANDBOX PROBE

bwrap 기능 테스트

--version 이 아니라 실제 --unshare-user 실행으로 판정. 불가 시 session_start 에 경고 1 회, 비활성.

PROFILES

strict / dev / eval

strict= 모두 승인, dev= 기본 (block), eval= 무인 (컨테이너 전제 + gate off).

19

위험 명령 패턴 (+ rm 대상 규칙)

29

게이트 테스트 (16 → 29)

bwrap x

현 환경 usersns 차단

컨테이너

= 집행 계층 (문서화)

안전 기본값 — v0.10.0 · v0.11.0

SE · NS Phase 가 바꾼 안전 기본값 — 전부 opt-out, env/settings 미설정 시 이전 동작과 바이트 동일

DEFAULTS CHANGED, ALL OPT-OUT

v0.10.0(SE 12 라운드): 커널 spawn env 에서 provider 키 제거 · 셀 30 분 wall-clock 타임아웃 + 커널 회수 · 게이트 7→19 패턴 + bash() 셀 · 보호 경로 검사 · v0.11.0(NS): env allowlist 모드 · rm 대상별 판정 + permissionGate.allow · permission_gate 텔레메트리 · BPE progress 원장 /recall(evo 게이트 뒤).
끄는 법: EVOPI_KERNEL_INHERIT_SECRETS=1 · kernel.cellTimeoutMs=0 · EVOPI_PERMISSION_GATE=warn|off · kernel.envPolicy(기본 denylist).

KERNEL ENV FILTER

provider 키 차단

kernel-env.ts: provider 키 30 종 + EVOPI_API_KEY_POOL_* 를 spawn env 에서 제거 (진단 tail 에 withheld 목록).
kernel.envPolicy=allowlist 면 PATH · HOME · 로케일 · XDG_* · EVOPI_* · Python 툴링 + envAllow 만 전달 · opt-out: EVOPI_KERNEL_INHERIT_SECRETS=1.

CELL TIMEOUT

30 분 wall-clock

kernel.cellTimeoutMs 기본 1,800,000ms. 초과 시 interrupt → 무응답이면 커널 폐기, 다음 셀이 스냅샷에서 복원 · 오류명 KernelCellTimeout. 해제: EVOPI_KERNEL_CELL_TIMEOUT_MS=0|off 또는 cellTimeoutMs=0.

PERMISSION GATE

19 패턴 + 보호 경로

rlm.bash()/os.popen/subprocess/pexpect 셀까지 검사, .env · .git · ~/.ssh · 키 파일 · auth.json 쓰기는 확인 요구 · rm -r* 은 /, ~, *, .., cwd 외부, sudo 만 위험 (A3). permissionGate.mode(block|warn|off) · allow[] 정규식, 전역 + 프로젝트 settings.

GATE TELEMETRY

permission_gate 로그

5 결정 (allowed-by-whitelist · warned · blocked · confirmed/denied-by-user) 을 {hazardKind, tool, mode, sha256 16hex} 로 세션 로그에 기록 · 명령 원문 미저장, 로깅 실패는 결정에 무영향 · 미해소: rm -rf .(cwd 자체) 통과 · 타임아웃 UX(A4).

30 + 1 접두

차단 provider 키 (S1)

30 min

셀 타임아웃 기본 (R1)

19 / 5 / 8

위험 패턴 / 셀 마커 / 보호 경로 (S2)

opt-out

3 env + 2 settings 키

07

Knowledge Layer

시스템 프롬프트 조립 · 하네스 주입 · 컴팩션

Knowledge Layer — 컨텍스트와 메모리

무엇이 시스템 프롬프트로 들어오는가

SYSTEM PROMPT ASSEMBLY

buildSystemPrompt(system-prompt.ts) 조립 순서 : RLM 프롬프트 (불변) → 서브에이전트 가이드 → 하네스 상태 (kind 별 6 개 · 180 자) → MCP 안내 → Additional Guidance → Project Context(AGENTS.md) → 스킬 목록 → APPEND_SYSTEM.md. evo on 이면 하네스 절단이 사전순 대신 MMR 선택기 (M17).

ALWAYS ON

AGENTS.md ·
SYSTEM.md

전역 + 프로젝트 .
_rebuildSystemPrompt 가 스킬 / 하네스
변경 시 재조립 (6 지점).

SKILLS

이름 + 설명만 상주

본문은 매칭 시 .Python 스킬은 import
가능 목록으로 노출 .

HARNESS

kind 별 요약 메뉴

+N more 로 넘침 표시 . 사전순 (기본)
또는 MMR(최근성 반감기 7 일 ×
jaccard 다양성 , 문자 예산).

MEMORY

mnemopi 병존

prime harness memory 와 저장소 분리 .
v1 은 선택기 (rerank) 로만 소비 , MCP
서버 겸용은 v2.

6 / kind

표시 상한 (저장 상한 아님)

180 chars

엔트리 요약 길이

MMR

harness.selection=mmr 또는 evo on

7 일

최근성 반감기

Context Window 관리

가득 차면 어떻게 되는가 — 요약 기반 컴팩션 (D5) 과 refine 연동

AUTO-COMPACTION

shouldCompact 가 한계 근접을 감지하면 findCutPoint 로 보존 구간을 정하고 generateSummary(compact 스킴) 가 요약을 만든다. 컴팩션 완료는 autoRefine 의 두 트리거 중 하나 (reason=compact) — 압축 직전의 궤적이 하네스 편집 후보가 된다.



summary

요약 기반 압축 (D5)

branch

브랜치 요약 (세션 트리)

/compact

수동 명령

compact→refine

압축이 진화의 신호

08

Execution & Multi-Agent

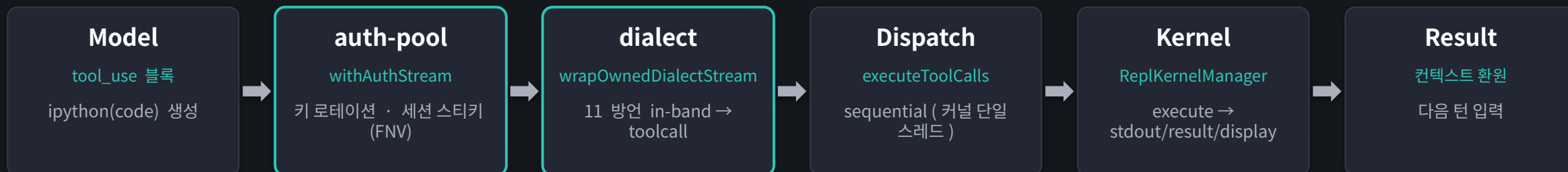
스트림 파이프라인 · 툴 디스패치 · rlm 서브에이전트

Execution Layer — 스트림 파이프라인

sdk.ts streamFn 클로저 하나에 omp 자산 두 개가 끼어든다

STREAM FN

prime 의 agent-loop 는 무수정 .sdk.ts:288 의 streamFn 이 ① getApiKeyAndHeaders ② EVOPI_API_KEY_POOL 이 있으면 withAuthStream (401→refresh 1 회 + 형제 전환 , 403/usage-limit→ 전 폴 순회 , replay-unsafe 경계 전만 무음 재시도) ③ streamSimple ④ dialect 가 켜지면 in-band 툴콜을 네이티브 toolcall 로 재물질화 .



AUTH-POOL

다중 키 로테이션

omp auth-retry 상태기계 (a/b/c) 이식 . Bun.hash → FNV-1a 순수 TS.

DIALECT

오픈모델 툴콜

hermes/qwen3/glm/kimi/deepseek 등 . off 면 스트림 reference 동일 (바이트 동일 게이트).

NATIVES

pi-natives prebuilt

natives-loader 가 leaf.node 직접 require(AVX2 선택), 실패 시 TS 폴백 .

0

prime agent-loop 수정

11 + 6

dialect/pool-stream 테스트

replay-safe

start 이벤트까지만 재시도

TS fallback

natives 미가용 플랫폼

Multi-Agent Layer — rlm 서브에이전트

서브에이전트가 Python 함수 호출이다

DELEGATE FROM THE KERNEL

모델이 셀 안에서 `rlm("...")` 을 호출하면 `host_request` 가 TS 로 올라가 `_createInlineRlmSubagentRuntime` 이 자식 `SessionManager + Agent` 를 `in-process` 로 띄운다 . 결과는 셀의 반환값으로 돌아오고 , 서브에이전트 명세는 하네스 `subagent kind` 로 영속된다 .

RLM()

커널 내 호출

`rlm.run / find_models /`
`list_subagents / delete_subagent — 4`
host handler.

ISOLATION

sub- 세션 아티팩트

`session-artifacts/<id>/sub-xxxxxxx/`.
부모 컨텍스트 오염 없음 (요약만 반환).

SPEC

subagent 엔트리

`refine` 이 만든 명세를 다음 세션에서
재사용 . Claude Code 의 `/agents` 에
대응 .

WORKTREES

v2 이연

`git worktree` 병렬 격리는 미구현 . `omp`
의 `worktree` 격리 이식이 백로그 .

in-process

자식 Agent 스폰 방식

host_request

TS↔Python 브리지

1024

dedup id 상한

v2

Worktrees

09

Observability & Integration

확장 훅 · 빌트인 확장 · MCP · 프로바이더

Observability Layer — Extension Hooks

31 개 라이프사이클 이벤트로 관찰 · 차단 · 교체

EXTENSION RUNNER

ExtensionRunner(runner.ts) 가 이벤트를 emit, 핸들러는 jiti 로 무컴파일 로드된 TS. tool_call 은 first-block 단락 (하나가 block 하면 즉시 차단), session_before_refine 은 {skip} 또는 {proposal} 로 refine 라운드를 억제하거나 플래너를 통째로 교체할 수 있다 .

session_*	start · before_switch · before_compact · before_refine · compact · shutdown · before_tree · tree
turn / agent	turn_start · turn_end · before_agent_start · agent_end
tool	tool_call(차단 가능) · tool_result · user_bash
model / ctx	before_provider_request · model_select · context · input

BUILTIN EXTENSIONS

- permission-gate — 항상 로드 (noExtensions 제외)
- grounded-refine — EVOPI_EVO=on 일 때만
- herdr-agent-state — HERDR_ENV=1 일 때만
- + 사용자 확장 : ~/.evopi/agent/extensions/, 프로젝트 .evopi/agent/extensions/
- + Background: rlm.bash 핸들 · scheduled-jobs.json · 백그라운드 refine 플랜

31

혹 이벤트

3

빌트인 확장

first-block

tool_call 단락

proposal

플래너 교체 계약

Integration Layer — MCP · Providers · Natives

외부 도구와 모델을 표준 계약으로 연결

INTEGRATION IN THE KERNEL

MCP 서버는 TS 가 아니라 커널 Python(rlm.mcp / mcp_base.py) 이 연결하고 도구를 함수로 노출한다 . 프로바이더는 pi-ai 의 열린 registerApiProvider 맵에 9 종이 등록되며 , node 전용 Bedrock 은 지연 import 로 브라우저 번들 검사를 통과한다 .

MCP	PROVIDERS	AUTH	NATIVES
rlm.mcp settings.mcp 등록 → McpManager 가 host handler 제공 → 커널이 서버 연결 · 도구 노출 . 종료 시 서버 정리 .	9 API 종 anthropic · openai(3) · azure · google · vertex · mistral · bedrock. Databricks 는 anthropic-messages + Bearer.	auth.json + OAuth provider 당 1 크리덴셜 (auth.json) + env 풀 . OAuth: Anthropic · Copilot · Codex(PKCE).	pi-natives 18.1.2 mmrRerank · cosinePairs · vectorTopK · diffLineRuns · nodeChainAt · blockBoundaries — 6 함수 스모크 통과 .

Python

MCP 클라이언트 위치

9

API 종

3

OAuth 프로바이더

6

natives 함수 (R6)

10

Evo Layer

Evo-Harness 논문 ↔ prime refine 델타 · grounded-refine · 평가

논문 ↔ prime 델타 판정 (R4)

EVO-HARNESS(arXiv 2608.15071) 9 단계 중 prime 에 없는 것만 v1 로 — 나머지는 v2

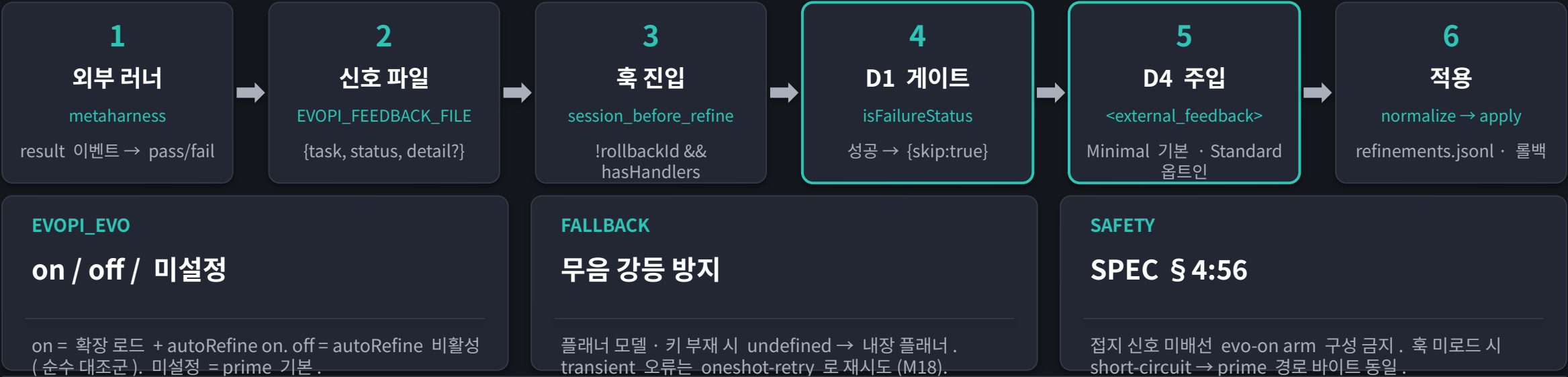
	논문 단계	prime 판정 evo.md	evopi v1 처리 DECISIONS R4
S1 저장 포맷	엔트리 = trigger+ 규칙 +evidence+scope, md+yaml	부분 — JSON 원장 , trigger/evidence 는 이벤트에만	v2 (D6)
S2 Select/Inject	관련성 기반 선택 ≤ b, 별도 모델	부분 — Inject 동일 , Select 없음 (사전순 6 개)	MMR 선택기 + 문자 예산 (M17, D8 백로그 ③ 선반영)
S3 컨텍스트	(x, τ, y, f) — 결과 · 피드백 슬롯	부분 — y/f 슬롯 없음	v1 D7 → D4 에 흡수
S4 Reflect	실패 / 부정 피드백 시에만	없음 — turn_interval/compact 트리거	v1 D1: 신호 성공이면 {skip}
S5 Evolver	ADD / MERGE / REVISE / SKIP	부분 — create/update/delete, MERGE · SKIP 없음	v2 (D5)
S6 2 레벨	general / topic 이중 컴파일	없음 — 단일 세션 1 패스	v2 (D2, 파일 수 경계)
S7 배치 · 예산	배치 16, 스킬 상한 5	없음 — 저장 상한 0 건	v2 (D9, D5 선행 필요)
S8 접지	환경 pass/fail 필수 (Self-Generated 는 악화)	없음 — 자가 판단 = Self-Generated 동형	v1 D4: <external_feedback> 주입
S9 평가	No-Evolve 대조군 , 3 회 평균	없음 (prime) / metaharness (omp)	v1 D0: 4-arm, autoRefine off = 대조군

grounded-refine — 접지 피드백으로 진화하기

논문 Table 4: Self-Generated 61.67 < No-Evolve 63.67 < Minimal 67.33 (SWE-bench Lite)

WHY GROUNDING

prime 의 refine 은 LLM 이 스스로 성공을 판단하는 Self-Generated 설정과 동형 — 논문상 No-Evolve 보다 나쁘다 .evopi 는 외부 pass/fail 신호를 session_before_refine 혹에서 읽어 (a) 신호 없음 → 무개입 (b) 성공 → skip (c) 실패 → 신호를 플래너 입력에 주입한다 . 접지 없는 evo-on 은 금지 (SPEC § 4).



+3.7pt

Minimal vs No-Evolve (논문)

1 file

grounded-refine.ts 208 줄

Minimal

기본 세밀도 (pass/fail)

off = prime

D7 대조군

평가 — metaharness 4-arm A/B

omp metaharness 를 bun 격리 (eval/) 로 재사용 , 코딩 트랙 kind:"edit"

EVAL IS OUTSIDE THE PRODUCT

제품은 node 전용 , 평가 러너는 구조적으로 bun 전용 (import.meta.dir, Bun.which...) → eval/ 소형 bun 워크스페이스로 격리 (R7). arm 은 잡네임 접두 규약 (evopi-<arm>), 피실험 CLI 는 bun overrides 로 @evopi/pi-coding-agent 리포인트 (Q2).

evopi-omp

상류 omp

게시된 @oh-my-pi/pi-coding-agent 18.1.2 그대로 — 대조 상류 .

evopi-prime

prime 스켈레톤

prime 계열 설정 대조군 .

evopi-evooff

evopi · evo off

EVOPI_EVO=off → autoRefine 비활성 .
순수 대조군 (No-Evolve).

evopi-evoon

evopi · evo on

EVOPI_EVO=on +
EVOPI_FEEDBACK_FILE 배선 필수 . 신호
= 러너 result 이벤트 .

SKIP

실 A/B — API 키 부재

2 smoke

mock completeSimple · D1 로직

edit

task_success · edit_success

3 회 평균

논문 프로토콜 (1.3) — 키 확보 시

11

비교와 점검

Claude Code · oh-my-pi · prime-agent · evopi

4 자 비교 매트릭스 (1/2)

제어 루프 · 도구 / 실행 · 컨텍스트 · 권한 / 안전

	Claude Code 참조 자료 기준	oh-my-pi omp v18.1.2	prime-agent v0.9.1	evopi v0.11.0
제어 루프	Gather-Act-Verify 독립 도구 병렬 실행	agent-loop 3,010 줄 · 29 톨 병렬 steering/aside/follow-up 3 큐 · TTSR	동일 agent-loop AgentSession 12k 줄이 감쌘	prime 루프 무수정 streamFn 에 dialect·pool 삽입
도구 / 실행	5 카테고리 tool_use 로컬 /Cloud VM/Remote	29 빌트인 톨 · hashline 편집 Rust in-process 셸 (brush) · xd:// 장치	ipython 단일 톨 IPython 커널 + dill 스냅샷	ipython + hashline_edit(선택) 커널 무변경 · natives prebuilt
컨텍스트	CLAUDE.md · MEMORY.md Skills 3 단 · 자동 압축	AGENTS.md · 메모리 백엔드 4 종 컴팩션 5 전략 (snapcompact PNG 포함)	AGENTS.md · Harness 원장 요약 컴팩션	prime + MMR 하네스 선택기 (mnemopi 커널) · 요약 컴팩션
권한 / 안전	4 모드 · allowlist 계층 체크포인트 되감기 · Hooks 게이트	approval allow/deny/prompt · 3 모드 COW worktree 격리 · OS 샌드박스 없음	권한 프롬프트 샌드박스는 예제 확장 (bwrap)	permission-gate 내장 (block/warn/off) + 샌드박스 프로브 (D3 폴백)

4 자 비교 매트릭스 (2/2)

확장성 · 자기개선 · 모델 연결 · 런타임 / 배포 · 평가

	Claude Code 참조 자료 기준	oh-my-pi omp v18.1.2	prime-agent v0.9.1	evopi v0.11.0
확장성	Skills · MCP · Hooks(14 이벤트) Subagents · Plugins	혹 28 · 슬래시 83 · 커스텀 툴 MCP · Advisor · 서브에이전트 task	extensions(jiti) 31 혹 Python 스킬 · MCP(커널) · rlm()	prime 표면 그대로 + 빌트인 3 확장
자기개선	MEMORY.md 갱신 (하네스 자동 정련 없음)	autolearn 자동 SKILL.md 저작 learn/manage_skill · 메모리 4 백엔드	continual harness refine (자가 판단 = Self-Generated)	prime refine + 접지 (D4) + 실패 한정 (D1) + MMR 선택
모델 연결	Claude (Sonnet/Opus) /model 전환	카탈로그 66 프로바이더 · dialect 11 폴 로테이션 · gateway/broker 사이드카	9 API · 카탈로그 32 프로바이더 OAuth 3 · Bedrock · auth.json 1 키 / 프로바이더	prime 카탈로그 + 폴 로테이션 + dialect 11 + Databricks
런타임 / 배포	(자료 미기재) npm CLI · 8+ 인터페이스	Bun 전용 (Bun.* 1,282 회) Rust napi 9 crate · 단일 바이너리 설치	node · uv Python install.sh 45KB · R2	node ≥22 · Bun 실패 0 curl sh → GitHub Pages
평가	(자료 미기재) Hooks 로깅 · /context	metaharness (harbor/edit/snapcompact)	없음 (autoRefine off 로 대조군만)	metaharness bun 격리 4-arm · 실 실행 SKIP(키)

evopi 하네스 점검 결과

강점 4 · 리스크 4 → 2 해소 (v0.10.0) — 코드 실측 (evopi-harness-inventory.md) 기준

STRENGTH 골격 무수정 prime agent-loop · 커널 무변경, 확장은 streamFn/ 혹은 빌트인 seam 에만 .tsgo 0 · Bun 실행코드 0 · .omp 0.	STRENGTH 대조군 내장 EVOPI_EVO=off 가 prime 원본과 바이트 동일 — A/B 설계가 코드에 박혀 있다 (D7).	STRENGTH 접지된 진화 논문상 유일하게 직접 절제 근거가 있는 델타 (D4) 를 1 파일로 배선. 안전 구속을 SPEC 에 명문화.	STRENGTH 이식 자산 활성화 dialect · auth-pool · mnemopi · hashline · oneshot-retry 전부 소비 배선 완료 (휴면 0).
RESOLVED · v0.10.0 커널 env 필터 kernel-env.ts 가 provider 키 30 종 + EVOPI_API_KEY_POOL_* 를 spawn env 에서 제거 (v0.11.0 allowlist 모드 추가). opt-out EVOPI_KERNEL_INHERIT_SECRETS → 슬라이드 22.	RISK · HIGH OS 샌드박스 미구현 bwrap 은 프로브만, 래핑 코드는 예제 확장에만. 집행 계층 = 컨테이너 경계 전제 — 배포 문서로만 보장.	RESOLVED · v0.10.0 셀 30 분 타임아웃 kernel.cellTimeoutMs 기본 30 분 — interrupt 무응답이면 커널 폐기 후 스냅샷 복원 (KernelCellTimeout). 남은 것: 타임아웃 UX(A4) → 슬라이드 22.	RISK · MED 실 A/B 미실행 evo 효과 주장은 아직 논문 수치 인용. 키 확보 후 4-arm 3 회 실행 전까지는 가설 (GAP-4).

기타 : TS bash/edit 툴 정의 미등록 (의도된 단일 툴 철학) · `prime/config.json` 읽기 2 곳은 외부 Prime CLI interop(승인된 예외) · RESULTS.md 의 "pi-ai mock 없음" 서술은 현재 providers/faux.ts 존재와 불일치 (문서 갱신 필요).

종합 플로우 — End to End

요청에서 검증된 결과, 그리고 하네스 갱신까지 한 흐름으로



Observability (Hooks)

31 이벤트로 전 과정 관찰 / 검증 / 차단 — tool_call(2) · turn_end(5) · session_before_refine(7) · session_compact



미완이면 다음 셀 결정하며 루프 반복 · refine 이 편집한 하네스는 다음 세션의 Gather 로 들어온다

Input→Output

단일 에이전틱 흐름

feeds

지식 + 하네스가 매 단계 공급

hooks

전 과정 관측 / 통제

grounded

검증 결과가 진화의 유일한 입력

핵심 요약

여덟 개 레이어 + 하나의 evo 레이어, 하나의 루프로 기억하기

CORE Agentic Loop Think → ipython → Verify. prime agent-loop 무수정.	INPUT 진입 / 세션 / 권한 6 인터페이스 · jsonl+dill 복원 · permission-gate · 안전 기본값 (opt-out).	KNOWLEDGE 컨텍스트 / 하네스 AGENTS.md · Skills · Harness 원장 (MMR) · 요약 컴팩션.	EXECUTION 커널 / 스트림 IPython 커널 · streamFn(auth- pool · dialect) · sequential.	EVO 접지된 진화 grounded-refine(D1+D4) · EVOPI_EVO · off = prime.
MULTI-AGENT rlm 서브에이전트 커널 함수 호출 → in-process 자식 Agent. Worktrees v2.	OBSERVABILITY Extension Hooks 31 이벤트 · 빌트인 3 · 플래너 교체 계약.	INTEGRATION MCP/Providers rlm.mcp(커널) · 9 API · OAuth 3 · natives 6.	OUTPUT Task Result 검증된 산출물 + refinements.jsonl 갱신.	EVAL metaharness bun 격리 4-arm. 실 실행은 키 확보 시.

1 loop

Think-Execute-Verify

1 tool

ipython — 나머지는 Python

4 kinds

하네스 원장 엔트리

evo optional

off = prime 원본 (D7)

접지 필수

Self-Generated 는 금지

References

공식 문서 · 원본 레포 · 논문 · 본 프로젝트 산출물

PAPER

EVO-HARNESS: Context-to-Harness Skill Compilation for Self-Evolving Agents — arXiv 2608.15071

S1-S9 메커니즘, Table 4 접지 피드백 절제, Appendix E/F 프롬프트 · 상수 — docs/analysis/evo.md 에 라인 인용

UPSTREAM

prime-agent (PrimeIntellect-ai) v0.9.1 · oh-my-pi v18.1.2 · pi (earendil-works)

골격 · TS 자산 공급원 · 원류 · 읽기 전용 분석: docs/analysis/prime.md, omp.md, *-master-arch.md

CLAUDE CODE

Claude Code Architecture (AWS Korea 최우형, 2026) · code.claude.com/docs · agentskills.io · modelcontextprotocol.io

8 레이어 맵 참조 시나리오 — docs/analysis/claude-code-arch.md (PDF 페이지 인용, 자료 미기재 항목 분리)

EVOPI

github.com/sunwoo95/oh-my-evopi · docs/design/{DECISIONS,PORTING,AUDIT-initial-goal}.md · docs/analysis/*.md

결정 원장 (D1-D8, R3-R10, SE/NS M- 항목) · 이식 등급표 · 코드 실측 인벤토리 (evopi-harness-inventory.md) · docs/diagrams/*.dot

Thank you!

oh-my-evopi · <https://github.com/sunwoo95/oh-my-evopi>

설치 : `curl -fsSL https://sunwoo95.github.io/oh-my-evopi/install.sh | sh`

실행 : `evopi` · `evo` 레이어 토글 : `EVOPI_EVO=on|off`